Quote PDF Auto-Generation Architecture

URLA SHOES — Salesforce Sandbox — Team Presentation Document

> **Critical design note:** This solution intentionally does **NOT** use an Apex trigger. Salesforce's native server-side PDF generation requires Visualforce (`PageReference.getContentAsPDF()`), which was explicitly out of scope. PDF rendering therefore runs **client-side** in an LWC using the jsPDF library, with automation driven by a polling mechanism instead of a record trigger.

1. Components

Layer	Component	Purpose
Browser	LWC `quotePdfGenerator`	Detects record changes via polling, renders PDF via jsPDF, calls Apex to upload.
Static Resource	`jsPDF` (UMD min, ~365 KB)	Client-side PDF library loaded into the LWC.
Apex Service	`QuotePdfService`	4 methods: change signature, fresh data fetch, version number, upload attachment.
SObject	`Quote` , `QuoteLineItem` , `Attachment`	Standard objects; PDFs land in Notes & Attachments.

2. End-to-End Flow

1 User opens the Quote record page

- LWC mounts → `connectedCallback()` runs
- `loadScript(jsPdfResource)` downloads/caches the library
- First `pollOnce()` stores a *baseline* change signature
- `setInterval(pollOnce, 3000)` starts the polling loop

force-app/main/default/lwc/quotePdfGenerator/quotePdfGenerator.js (connectedCallback)

2 User clicks "Add Products" and saves line items

- Salesforce inserts `QuoteLineItem` records
- Quote rollup fields update server-side: `Subtotal` , `Discount` , `Tax` , `TotalPrice` , `GrandTotal`
- `Quote.LastModifiedDate` advances
- The browser knows nothing yet

3 Polling detects the change (within ~3 s)

```
async pollOnce() {
    const signature = await getQuoteChangeSignature({ quoteId: this.recordId });
    if (this.previousSignature && this.previousSignature !== signature) {
        this.scheduleRegenerate();   // ← TRIGGER POINT
    }
    this.previousSignature = signature;
}
```

The Apex method returns a compact pipe-delimited string composed of 7 parts:

```
@AuraEnabled
public static String getQuoteChangeSignature(Id quoteId) {
    Quote q = [SELECT LastModifiedDate, Subtotal, Discount, Tax,
                      TotalPrice, GrandTotal FROM Quote WHERE Id=:quoteId];
    Integer lineCount = [SELECT COUNT() FROM QuoteLineItem WHERE QuoteId=:quoteId];
    return q.LastModifiedDate.getTime() + '|' + q.Subtotal + '|' + q.Discount
         + '|' + q.Tax + '|' + q.TotalPrice + '|' + q.GrandTotal
         + '|' + lineCount;
}
```

QuotePdfService.cls — getQuoteChangeSignature

4 Debounce (1.5 s wait)

```
scheduleRegenerate() {
    if (this.regenerateTimeoutId) clearTimeout(this.regenerateTimeoutId);
    this.regenerateTimeoutId = setTimeout(async () => {
        await this.handleGenerate();
    }, 1500);
}
```

Why debounce? The Add Products wizard can produce several DMLs in quick succession. The debounce waits until 1.5 s after the *last* save, so adding 5 products produces **one** PDF rather than five.

5 Actual generation — `handleGenerate()`

```
async handleGenerate() {
    this.isGenerating = true;
    // 1. Pull the freshest data (cacheable=false in Apex)
    const freshData = await getQuoteData({ quoteId: this.recordId });
    // 2. Compute the next version number
    const version = await getNextVersionNumber({ quoteId: this.recordId });
    // 3. Render the PDF in the browser via jsPDF
    const base64 = this.buildPdfBase64(freshData, version);
```

```
    // 4. Hand off to Apex which inserts the Attachment
    const fileName = `${freshData.quote.QuoteNumber}_v${version}.pdf`;
    await uploadPdf({ quoteId: this.recordId, base64Body: base64, fileName });
    // 5. Toast the user
    this.dispatchEvent(new ShowToastEvent({ title:'PDF generated', ... }));
}
```

6 PDF rendering with jsPDF (in the browser)

`buildPdfBase64(data, version)` draws the invoice layout that matches the design template:

- Top left: bold **INVOICE** title (46 pt)
- Top right: **URLA SHOES CORP.** + address + email + phone
- Meta row: Invoice #, Due Date, Invoice date, Prepared by
- Pink banner: **CUSTOMER DETAILS** + Name / Address / Phone / Email
- Line item table with pink header: *DESCRIPTION / QTY / PRICE / TOTAL*. Description falls back to `Product2.Name` when blank.
- Bottom left: Terms & Conditions paragraph
- Bottom right: SUBTOTAL / DISCOUNT / TAXES / GRAND TOTAL
- Page footer: Signature / Name / Date lines

Output: PDF document → base64 string handed to Apex.

7 Apex inserts the Attachment

```
@AuraEnabled
public static Id uploadPdf(Id quoteId, String base64Body, String fileName) {
    Attachment a = new Attachment(
        ParentId    = quoteId,
        Name        = fileName,                    // e.g. "00000006_v2.pdf"
        Body        = EncodingUtil.base64Decode(base64Body),
        ContentType = 'application/pdf'
    );
    insert a;
    return a.Id;
}
```

The new PDF appears in the **Notes & Attachments** related list immediately.

8 The next polling cycle updates the baseline

After the generation completes, the next `pollOnce` sees the now-current signature and stores it as the new baseline. This means **one save produces exactly one PDF** — no infinite loop.

3. Versioning Logic

```
@AuraEnabled
public static Integer getNextVersionNumber(Id quoteId) {
    Integer maxVersion = 0;
    for (Attachment a : [SELECT Name FROM Attachment
                         WHERE ParentId=:quoteId AND ContentType='application/pdf']) {
        Integer v = parseVersion(a.Name);   // regex: _v(\d+)\.pdf
        if (v > maxVersion) maxVersion = v;
    }
    return maxVersion + 1;
}
```

Scan all existing PDF attachments for the highest `_vN.pdf` suffix and return `N + 1`. First save → v1, second → v2, and so on.

4. Latency Budget

Stage	Duration
User clicks Save → DB committed	~0.5 s
Next polling cycle (worst case)	3 s
Debounce window	1.5 s
PDF render + upload	~0.3 s

Worst-case total ≈ 5 seconds from Save to PDF in Notes & Attachments

5. Key Architectural Decisions

> **5.1 — Why polling instead of an Apex trigger?**
>
> With Visualforce off the table, server-side PDF rendering is not possible. Generation has to happen in the browser via jsPDF, which means the automation lives inside an LWC. `getRecord` (Lightning Data Service) was tried first but does not consistently fire on QuoteLineItem rollup updates. Polling the Apex signature every 3 s is the most reliable detection mechanism.

> **5.2 — Why `cacheable=false` on `getQuoteData`?**
>
> Earlier versions had `cacheable=true`. Because the LWC fetches data right after a save, the cache returned stale (empty) data and the PDF showed no line items. Removing the cache forces every generation to hit fresh data — this was the root cause of the empty-PDF issue we hit during development.

> **5.3 — Why 3 s polling, why 1.5 s debounce?**
>
> 3 seconds is short enough to feel near-instant for a user; long enough that the per-minute Apex calls (~20) are negligible. The 1.5 s debounce collapses wizard-driven DML bursts (e.g. Add Products inserting 5 line items) into a single PDF instead of five.

> **5.4 — Why Notes & Attachments rather than Files?**
>
> The original requirement explicitly named "Notes & Attachments". The classic `Attachment` sObject lands there and matches the requirement verbatim. `ContentDocument` (Files) would be the more modern choice and is a one-line swap if the team prefers it later.

6. Failure Modes

Scenario	Outcome
jsPDF static resource fails to load	Card shows red error: "*jsPDF library failed to load*". Polling never starts.
`getQuoteData` throws	Toast: "*PDF generation failed*" with the error message; user can click "Regenerate Now".
`uploadPdf` blocked by FLS or sharing	Toast with the Apex error; the PDF is regenerated next polling cycle if conditions change.

Two generations attempted concurrently	`isGenerating` guard returns early; only the first completes.
Network interruption during polling	Silent failure — the next 3-second poll retries automatically.
User makes 3 saves during the debounce	Each save resets the 1.5 s timer; only one PDF is produced after the last save.

7. File Layout

```
force-app/main/default/
├── classes/
│   ├── QuotePdfService.cls              ╳— 4 Apex methods
│   └── QuotePdfServiceTest.cls          ╳— 6 unit tests, 100% coverage
├── lwc/
│   └── quotePdfGenerator/
│       ├── quotePdfGenerator.js         ╳— main logic (polling, render, upload)
│       ├── quotePdfGenerator.html       ╳— UI template
│       ├── quotePdfGenerator.css        ╳— styling
│       └── quotePdfGenerator.js-meta.xml ╳— Quote record page target
├── staticresources/
│   ├── jsPDF.js                         ╳— jsPDF 2.5.1 UMD min (365 KB)
│   └── jsPDF.resource-meta.xml
└── settings/
    └── Quote.settings-meta.xml          ╳— enables the Quote object
```

8. Three Key Takeaways for the Team

1. **"Without Visualforce, automated PDF generation moves to the browser."** — modern, client-side, no server-side rendering dependency.
2. **"The trigger isn't an Apex trigger — it's a 3-second poll + 1.5-second debounce."** — simple, predictable, and reliable across record types of changes.
3. **"`cacheable=false` + fresh-fetch on demand = correct PDF every time."** — cacheable Apex was the cause of the early empty-PDF bug; the fix was deliberate and worth flagging.

Prepared for team review — URLA SHOES Salesforce Sandbox